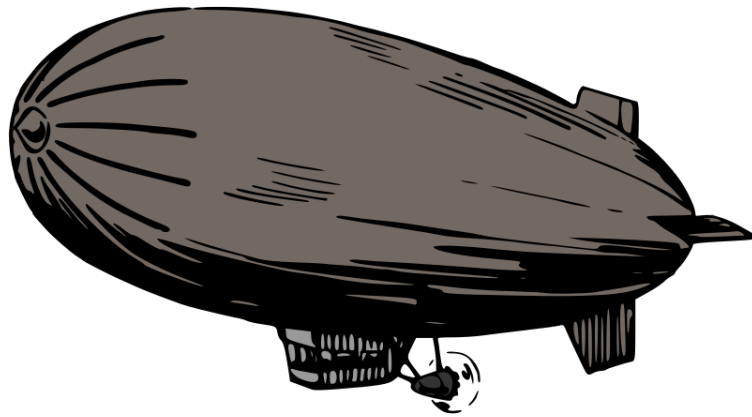


**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
THE UNIVERSITY OF TEXAS AT ARLINGTON**

**DETAILED DESIGN SPECIFICATION
CSE 4317: SENIOR DESIGN II
FALL 2025**



**AIRSHIP CREW
PROJECT AIRSHIP**

**ARSHDEEP SINGH
LAUREN ALLBRIGHT
CHRISTIAN MCCLAIN
BRANDON GARIBAY
AHMED OMAR**

REVISION HISTORY

Revision	Date	Author(s)	Description
0.1	8.30.2025	AS	document creation
1.0	8.30.2025	AS, LA, CM, BG, AO	Complete Document 1.0

CONTENTS

1	Introduction	5
2	System Overview	5
3	Ground Station Layer Subsystems	6
3.1	Layer Hardware	6
3.2	Layer Operating System	6
3.3	Layer Software Dependencies	6
3.4	Ground Station Subsystem 1 - Flask App Subsystem	6
3.5	Ground Station Subsystem 2 - Maps API Subsystem	7
3.6	Ground Station Subsystem 3 - Front End	8
3.7	Ground Station Subsystem 4 - Network Interface	9
4	Raspberry Pi Layer Subsystems	11
4.1	Layer Hardware	11
4.2	Layer Operating System	11
4.3	Layer Software Dependencies	11
4.4	RASPBERRY PI SUBSYSTEM 1 - RASPBERRY PI CAMERA	11
4.5	RASPBERRY PI SUBSYSTEM 2 - Camera Vision	11
4.6	RASPBERRY PI SUBSYSTEM 3 - Flask App	13
4.7	RASPBERRY PI SUBSYSTEM 4 - Network Interface	14
4.8	RASPBERRY PI SUBSYSTEM 5 - PID	15
4.9	RASPBERRY PI SUBSYSTEM 6 - Main Controller	16
4.10	Subsystem 7: GPS/Auxiliary Sensors	17
4.11	RASPBERRY PI SUBSYSTEM 8 - Serial Interface	18
5	Arduino Layer Subsystems	21
5.1	Layer Hardware	21
5.2	Layer Operating System	21
5.3	Layer Software Dependencies	21
5.4	Subsystem 1 - Serial Interface	21
5.5	Subsystem 2: IMU & Lidar Readings	22
5.6	Subsystem 3 - Power Gauge / Current Sensor	23
5.7	Subsystem 4 - Command Interpreter	24
5.8	Subsystem 5: Timer/Interrupt Controller	25
5.9	Subsystem 6 - Motor Control	26
6	Appendix A	28

LIST OF FIGURES

1	System architecture	5
2	Ground Station Flask App	6
3	Maps API	7
4	Front End	8
5	Network Interface	9
6	Example subsystem description diagram	12
7	Example subsystem description diagram	13
8	Flask App inside the Raspberry Pi Layer	14
9	Network interface inside the Raspberry Pi Layer	15
10	Example subsystem description diagram	15
11	Main Controller in the Raspberry Pi Layer	17
12	Example subsystem description diagram	18
13	Serial Interface in the Raspberry Pi Layer	19
14	Serial Interface in teh Arduino Layer	21
15	IMU & Lidar Readings subsystem diagram	22
16	Example subsystem description diagram	23
17	Command Interpreter subsystem in the Arduino Layer	24
18	Timer/Interrupt subsystem diagram	25
19	Motor Control subsystem in the Arduino Layer	26
20	Blimp CAD Design	28

LIST OF TABLES

1 INTRODUCTION

Project Airship is an environmentally friendly logistics solution designed to provide an efficient solution to air transport of cargo over short to medium distances. Project Airship is designed to navigate autonomously using GPS navigation and is equipped with helium-based lift and propulsion systems that allow this project to be more eco friendly and smart than traditional logistic transportation systems like planes and trucks. The requirements and specifications of this project are listed in the System Requirement Specification document and the structural design of the airship is explained in the Architectural Design Specification document. This document, the Detailed Design Specification, is intended to inform the reader in depth about the inner workings of the autonomous cargo blimp and the layers and subsystems it consists of.

2 SYSTEM OVERVIEW

The autonomous cargo blimp system combines hardware, onboard processing, communication, and ground station components to ensure safe and intelligent flight. Sensors like a PiCamera, IMU, and GPS gather live environmental data, while the Raspberry Pi processes this data locally, performing tasks such as edge detection and flat ground detection through computer vision to find suitable landing spots. A Flask-based server manages data flow and acts as the communication link between the airship and the operator interface. Live camera streams and sensor data are transmitted wirelessly to the ground station, where a GUI enables users to monitor the system and send commands. This modular setup guarantees real-time control, autonomy, and expandability, allowing the blimp to navigate and land safely in various testing situations.

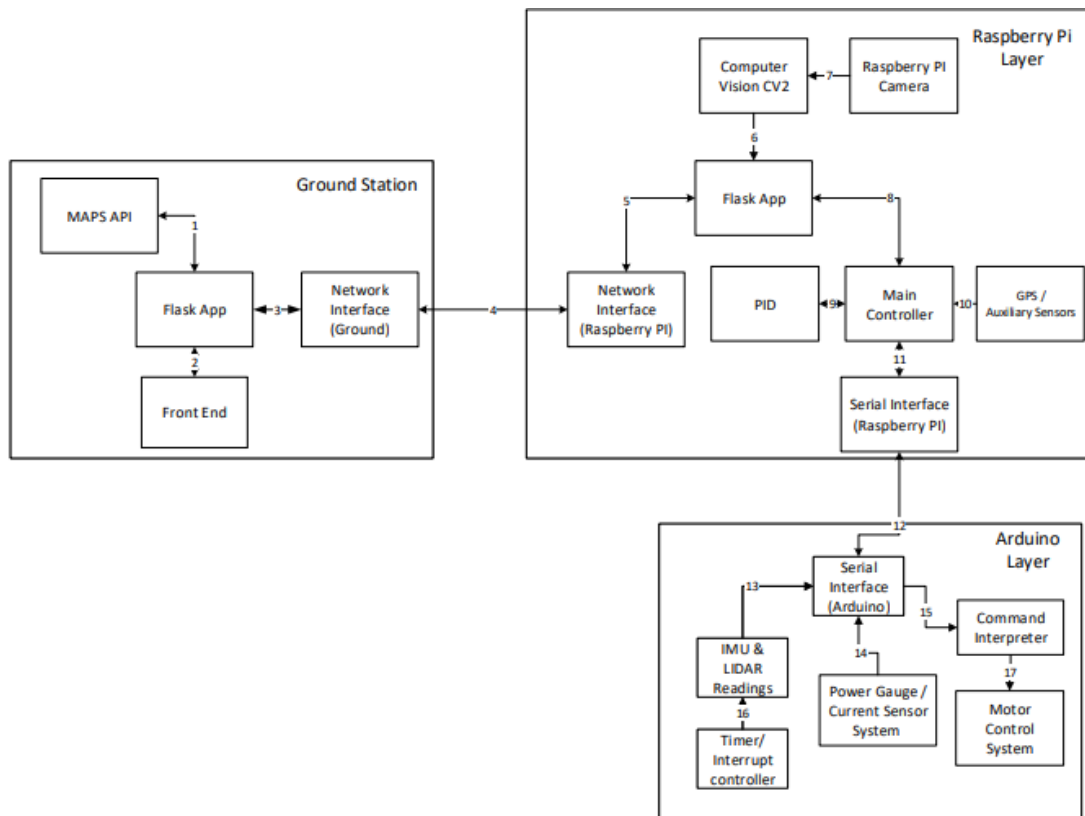


Figure 1: System architecture

3 GROUND STATION LAYER SUBSYSTEMS

In this section, the layer is described in terms of the hardware and software design. Specific implementation details, such as hardware components, programming languages, software dependencies, operating systems, etc. should be discussed. Any unnecessary items can be omitted (for example, a pure software module without any specific hardware should not include a hardware subsection). The organization, titles, and content of the sections below can be modified as necessary for the project.

This layer describes the operations and subsystems that run on the Ground Station computer. The Ground Station acts as the human operator interface. It is responsible from visualizing telemetry data, providing input commands, displaying real-time location on a map, and managing communication between the user and the Raspberry Pi layer.

3.1 LAYER HARDWARE

The hardware that is shared across this layer is a laptop with web browser access.

3.2 LAYER OPERATING SYSTEM

OS: Windows 11 (64-bit)

3.3 LAYER SOFTWARE DEPENDENCIES

- Python 3.x
- Flask Python Library
- Leaflet Maps
- Web Browser(Chrome, Edge, or Firefox)

3.4 GROUND STATION SUBSYSTEM 1 - FLASK APP SUBSYSTEM

This subsystem is the server-side application running on the Ground Station. It provides the backend services for the user interface, storing telemetry, and routing user commands to the airship.

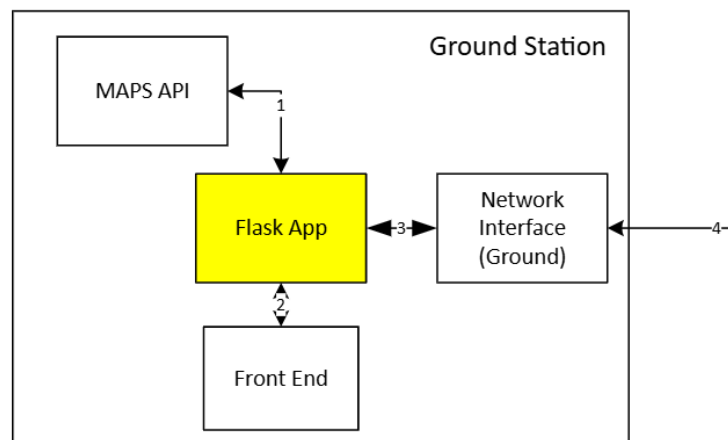


Figure 2: Ground Station Flask App

3.4.1 SUBSYSTEM HARDWARE

- Ground Station Laptop

3.4.2 SUBSYSTEM OPERATING SYSTEM

- Windows 11 (64-bit)

3.4.3 SUBSYSTEM SOFTWARE DEPENDENCIES

- Flask Python Library
- Request Python Library
- JSON Python Library

3.4.4 SUBSYSTEM PROGRAMMING LANGUAGES

- Python 3

3.4.5 SUBSYSTEM DATA STRUCTURES

- JSON Objects
- Python Dictionaries

3.4.6 SUBSYSTEM DATA PROCESSING

This subsystem acts as a data relay. It receives telemetry from the Raspberry Pi Flask App, processes it into JSON objects, and serves it to the frontend via API routes. It also receives commands from the frontend and forwards them to the Raspberry Pi.

3.5 GROUND STATION SUBSYSTEM 2 - MAPS API SUBSYSTEM

This subsystem integrates a mapping service to display the airship's location, waypoints, and flight paths. It allows the operator to interact with the map by placing new waypoints, which are then transmitted to the Flask App and forwarded to the airship.

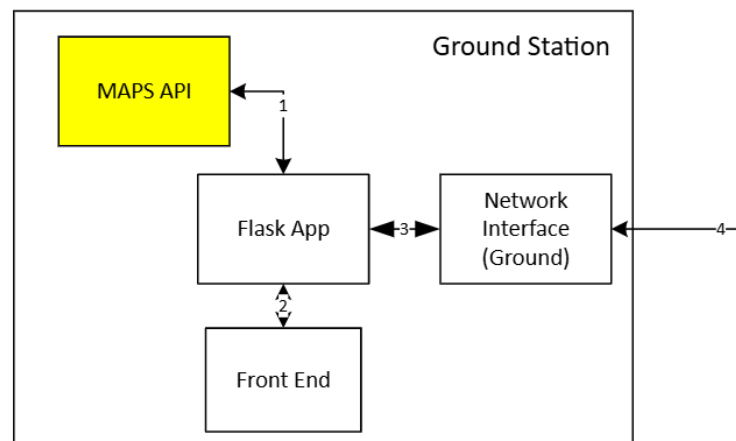


Figure 3: Maps API

3.5.1 SUBSYSTEM HARDWARE

- Ground Station Laptop

3.5.2 SUBSYSTEM OPERATING SYSTEM

- Windows 11 (64-bit)

3.5.3 SUBSYSTEM SOFTWARE DEPENDENCIES

- Leaflet.js
- OpenStreetMap

3.5.4 SUBSYSTEM PROGRAMMING LANGUAGES

- JavaScript
- HTML

3.5.5 SUBSYSTEM DATA STRUCTURES

- JSON Objects
- JavaScript Arrays

3.5.6 SUBSYSTEM DATA PROCESSING

This subsystem dynamically updates the displayed airship position and connected flight paths on the map. It converts user clicks into waypoint coordinates, packages them as JSON, and sends them through the frontend to the Flask backend.

3.6 GROUND STATION SUBSYSTEM 3 - FRONT END

This subsystem provides the user interface for monitoring and controlling the airship. It runs in a web browser and includes real-time telemetry display, a video feed viewer, and interactive map integration.

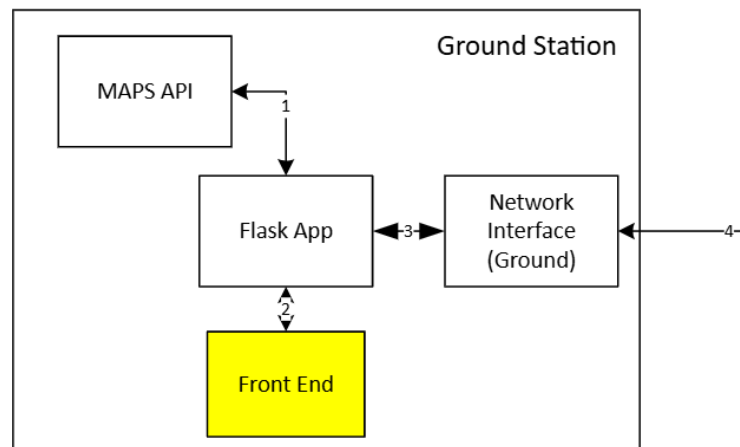


Figure 4: Front End

3.6.1 SUBSYSTEM HARDWARE

- Ground Station Laptop

3.6.2 SUBSYSTEM OPERATING SYSTEM

- Windows 11 (64-bit)

3.6.3 SUBSYSTEM SOFTWARE DEPENDENCIES

- HTML5/CSS3
- JavaScript

3.6.4 SUBSYSTEM PROGRAMMING LANGUAGES

- JavaScript
- HTML
- CSS

3.6.5 SUBSYSTEM DATA STRUCTURES

- JSON Objects
- JavaScript Objects

3.6.6 SUBSYSTEM DATA PROCESSING

This subsystem displays telemetry data such as altitude, speed, GPS location, and system status in real time. It also displays video streaming from the Raspberry Pi camera and provides controls for sending commands.

3.7 GROUND STATION SUBSYSTEM 4 - NETWORK INTERFACE

This subsystem provides the user interface for monitoring and controlling the airship. It runs in a web browser and includes real-time telemetry display, a video feed viewer, and interactive map integration.

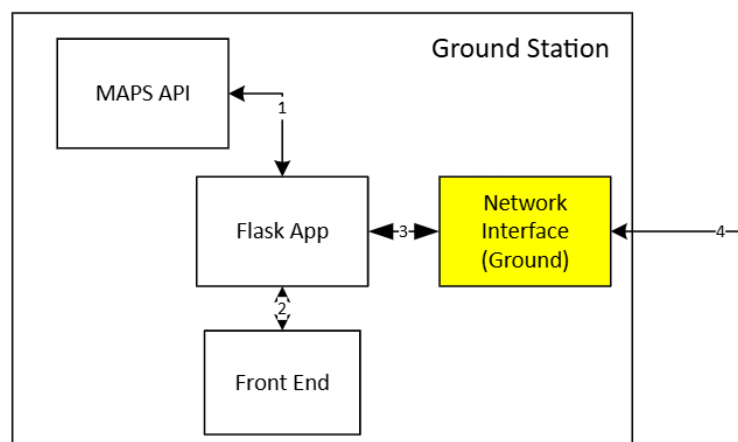


Figure 5: Network Interface

3.7.1 SUBSYSTEM HARDWARE

- Ground Station Laptop
- Ground Station WiFi Adapter (internal or USB)

3.7.2 SUBSYSTEM OPERATING SYSTEM

- Windows 11 (64-bit)

3.7.3 SUBSYSTEM SOFTWARE DEPENDENCIES

- Windows Network Stack
- Browser WebSocket

3.7.4 SUBSYSTEM PROGRAMMING LANGUAGES

- JavaScript
- Python

3.7.5 SUBSYSTEM DATA STRUCTURES

- JSON Objects

3.7.6 SUBSYSTEM DATA PROCESSING

This subsystem does not perform data processing but ensures reliable network connectivity between the Ground Station Flask App and the Raspberry Pi Flask App.

4 RASPBERRY PI LAYER SUBSYSTEMS

This layer describes the operations and subsystems on the Raspberry Pi. This layer acts like the brain of the airship, allowing complex actions to be taken independently of the ground station and allowing auto navigational capabilities. The role of the Raspberry Pi layer is to communicate with the Ground Station and the Arduino Layers and make control decisions regarding the airship's movement.

4.1 LAYER HARDWARE

The hardware that is shared across the entire layer is the: Raspberry Pi 4B 4GB model. This computer will be used by all subsystems.

4.2 LAYER OPERATING SYSTEM

OS: Raspberry Pi OS lite (64 - bit). This operating system will be used to run the subsystems on this layer.

4.3 LAYER SOFTWARE DEPENDENCIES

Software Dependencies:

- Python Interpreter
- PIP

4.4 RASPBERRY PI SUBSYSTEM 1 - RASPBERRY PI CAMERA

This subsystem uses the PiCamera module attached to the Raspberry Pi to record real-time video footage. It provides the main visual input for the computer vision and streaming systems. Both the camera vision module and the Flask App access this camera feed.

4.4.1 SUBSYSTEM HARDWARE

- Raspberry Pi Camera

4.4.2 SUBSYSTEM OPERATING SYSTEM

- Raspberry Pi OS (Lite)

4.4.3 SUBSYSTEM SOFTWARE DEPENDENCIES

- Picamera2
- OpenCV

4.4.4 SUBSYSTEM PROGRAMMING LANGUAGES

- Python3

4.4.5 SUBSYSTEM DATA STRUCTURES

NONE

4.4.6 SUBSYSTEM DATA PROCESSING

NONE

4.5 RASPBERRY PI SUBSYSTEM 2 - CAMERA VISION

This subsystem performs real-time computer vision processing to detect safe, flat landing zones using the Raspberry Pi camera feed. It is implemented in Python and runs as a local software module on the Raspberry Pi.

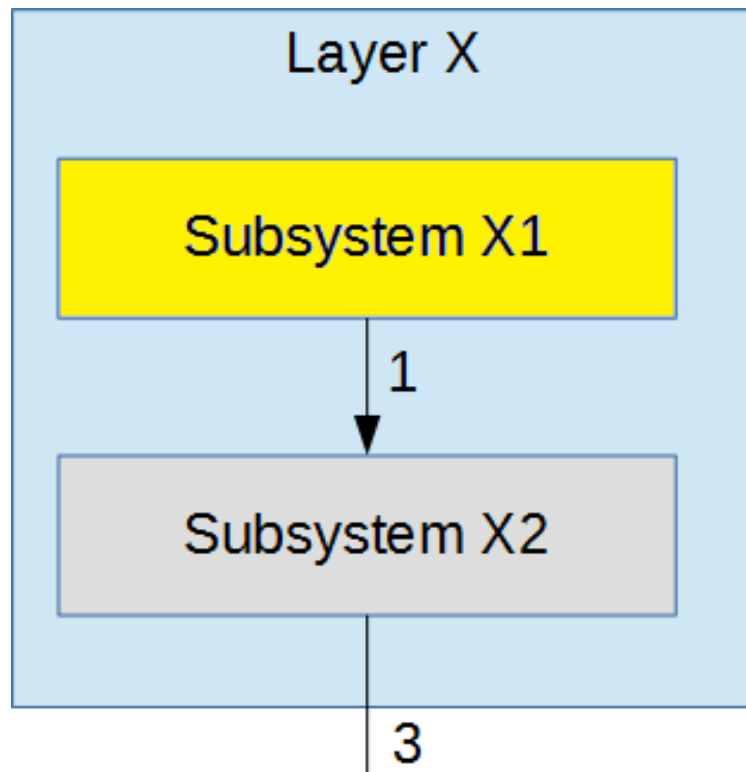


Figure 6: Example subsystem description diagram

4.5.1 SUBSYSTEM HARDWARE

- PiCamera Module v2

4.5.2 SUBSYSTEM OPERATING SYSTEM

- Raspberry Pi OS (Lite)

4.5.3 SUBSYSTEM SOFTWARE DEPENDENCIES

- OpenCV
- numpy
- PiCamera

4.5.4 SUBSYSTEM PROGRAMMING LANGUAGES

- Python3

4.5.5 SUBSYSTEM DATA STRUCTURES

- OpenCV Image Frames
- Contours
- Filtered Output

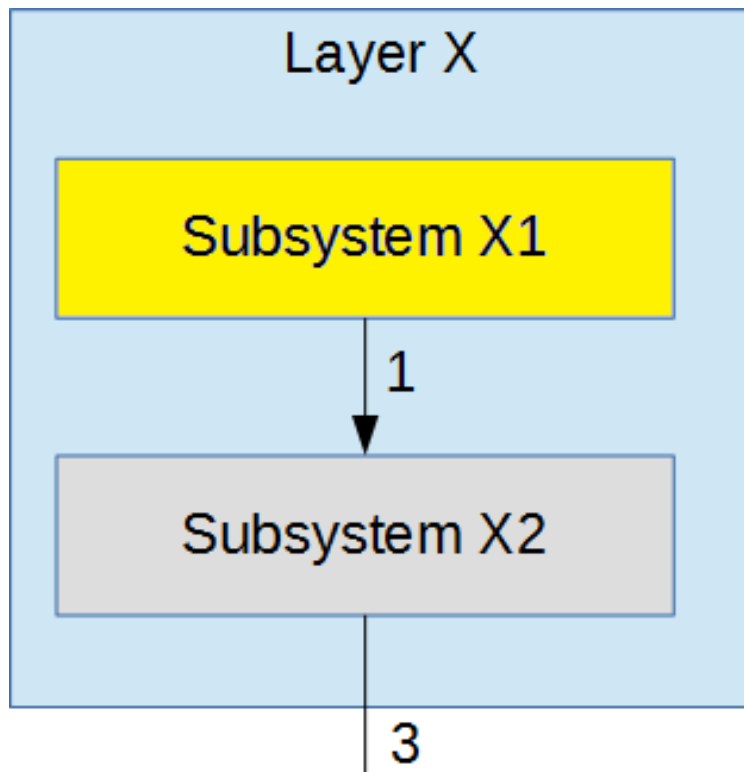


Figure 7: Example subsystem description diagram

4.5.6 SUBSYSTEM DATA PROCESSING

This subsystem uses the Canny Edge Detection algorithm to identify strong edges in grayscale video frames. The processed frames are then passed to OpenCV's contour detection, which locates continuous curves and determines their area.

4.6 RASPBERRY PI SUBSYSTEM 3 - FLASK APP

This subsystem is the web-app portion of the Raspberry Pi layer. It creates routes for data communication, storage, and transfer. The Flask App is the primary communicator with the ground-station; providing routes for accessing data, sending commands, and viewing the video feed.

4.6.1 SUBSYSTEM HARDWARE

NONE

4.6.2 SUBSYSTEM OPERATING SYSTEM

NONE

4.6.3 SUBSYSTEM SOFTWARE DEPENDENCIES

Software Dependencies:

- Flask Python Library
- Time Python Library
- cv2 Python Library

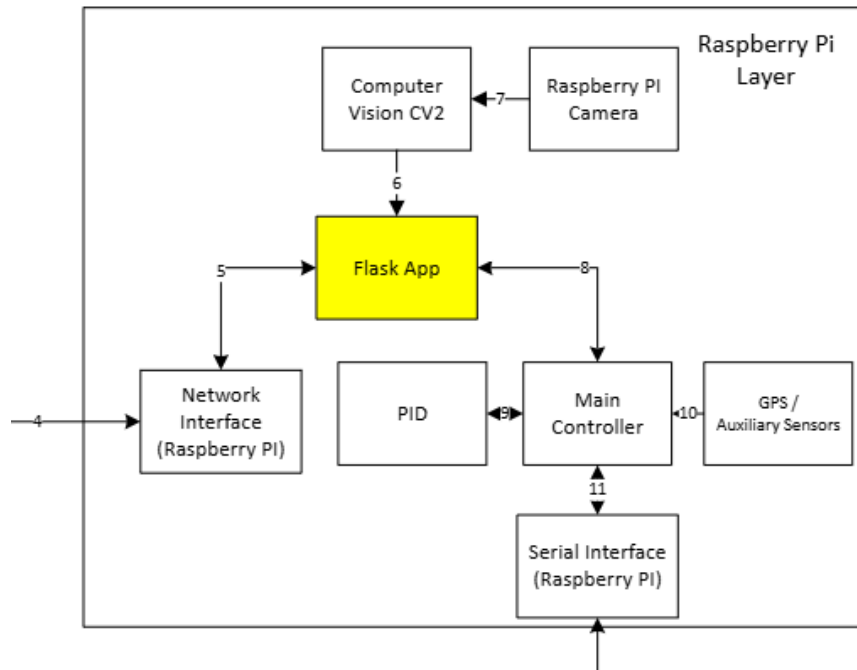


Figure 8: Flask App inside the Raspberry Pi Layer

4.6.4 SUBSYSTEM PROGRAMMING LANGUAGES

This subsection is created in Python and thus requires the Python Programming language to function.

4.6.5 SUBSYSTEM DATA STRUCTURES

The data structures used by this subsystem:

- JSON Objects
- Python Dictionaries

4.6.6 SUBSYSTEM DATA PROCESSING

NONE. This layer only sends, receives, and stores data.

4.7 RASPBERRY PI SUBSYSTEM 4 - NETWORK INTERFACE

This is the network interface on the Raspberry Pi, which will allow us to create a network access point on the Pi and allow the ground station to connect to the Raspberry Pi.

4.7.1 SUBSYSTEM HARDWARE

BrosTrend 650Mbps Linux Compatible WiFi Adapter

4.7.2 SUBSYSTEM OPERATING SYSTEM

Raspberry Pi OS

4.7.3 SUBSYSTEM SOFTWARE DEPENDENCIES

Network Manager

4.7.4 SUBSYSTEM PROGRAMMING LANGUAGES

NONE

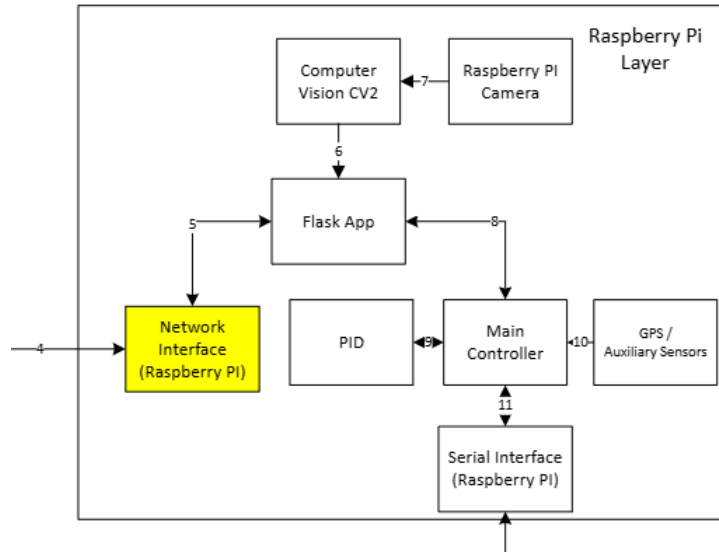


Figure 9: Network interface inside the Raspberry Pi Layer

4.7.5 SUBSYSTEM DATA STRUCTURES

NONE

4.7.6 SUBSYSTEM DATA PROCESSING

NONE

4.8 RASPBERRY PI SUBSYSTEM 5 - PID

This subsystem is responsible for creating an output that will speed up or slow down the motors to a speed that will maintain a set altitude for the blimp. This subsystem will take sensor data as input and calculate the speed of the upward motors.

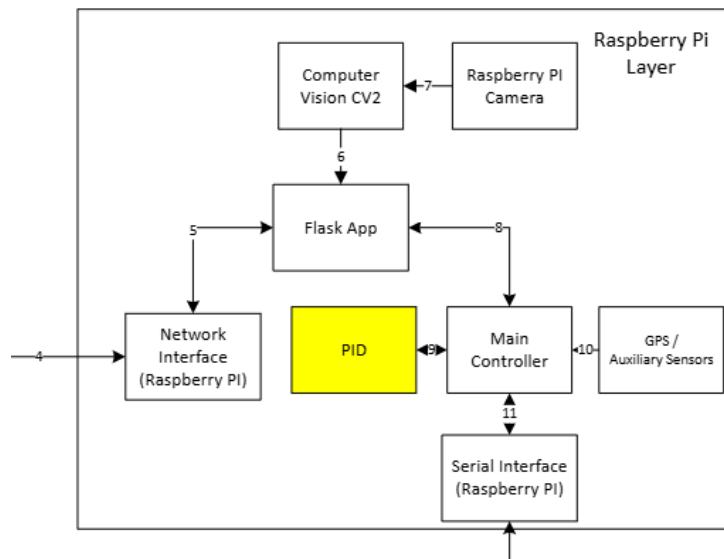


Figure 10: Example subsystem description diagram

4.8.1 SUBSYSTEM HARDWARE

NONE

4.8.2 SUBSYSTEM OPERATING SYSTEM

NONE

4.8.3 SUBSYSTEM SOFTWARE DEPENDENCIES

Software Dependencies:

- Time Python Library
- Threading Python Library
- Queue Python Library

4.8.4 SUBSYSTEM PROGRAMMING LANGUAGES

Python programming language

4.8.5 SUBSYSTEM DATA STRUCTURES

The data structures used by this subsystem:

- Python Dictionaries
- Python Variables

4.8.6 SUBSYSTEM DATA PROCESSING

PID controller algorithm with all three proportional, derivative, and integral components. These components and the controller itself are implemented in the standard way, with values being stored in the PID dictionary that are initialized on startup.

4.9 RASPBERRY PI SUBSYSTEM 6 - MAIN CONTROLLER

The Main Controller is the subsystem that is responsible for starting and stopping other subsystems in the Raspberry Pi Layer as well as facilitating communication between them. However, the primary responsibility of the Main Controller is to implement the auto-navigation algorithm that allows the blimp to move from the source to the destination.

4.9.1 SUBSYSTEM HARDWARE

NONE

4.9.2 SUBSYSTEM OPERATING SYSTEM

NONE

4.9.3 SUBSYSTEM SOFTWARE DEPENDENCIES

Software Dependencies:

- Time Python Library
- Threading Python Library
- Flaskapp Python Library

4.9.4 SUBSYSTEM PROGRAMMING LANGUAGES

Python Programming Language

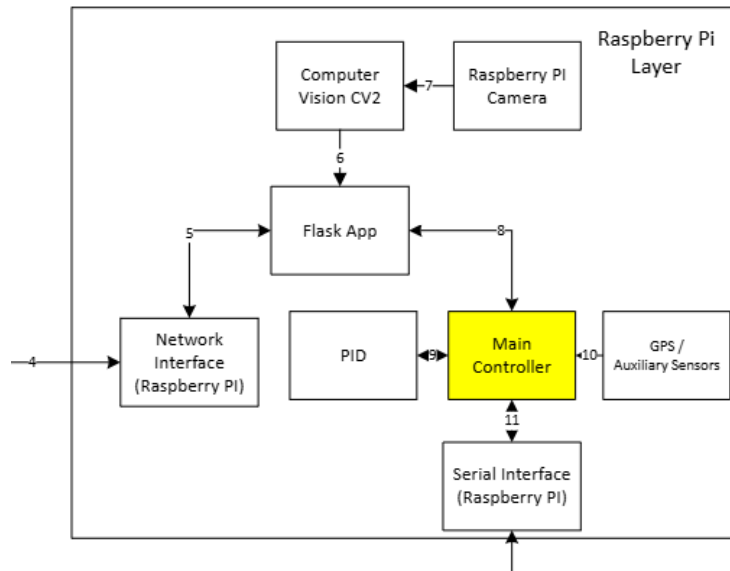


Figure 11: Main Controller in the Raspberry Pi Layer

4.9.5 SUBSYSTEM DATA STRUCTURES

- waypoints
- Flaskapp blimp data
- Flaskapp control commands
- blimp data lock
- pid controller

4.9.6 SUBSYSTEM DATA PROCESSING

The Main Controller handles real-time data from GPS and Arduino sensors using multi threaded functions that share information through synchronized dictionaries. It uses a PID control algorithm to adjust motor speeds and maintain the desired altitude based on sensor feedback. The navigation logic applies the Haversine formula and bearing calculations to determine the blimp's position and heading relative to a list of waypoints, then sends rudder and motor commands to reach each one. All subsystems run simultaneously to enable smooth autonomous flight and communication with the ground station via Flask.

4.10 SUBSYSTEM 7: GPS/AUXILIARY SENSORS

This hardware subsystem acquires precise telemetry data, GPS coordinates, and other aviation data to report the flight status of the blimp. The data is collected via global satellites to ensure accurate readings using the GPS hat device.

4.10.1 SUBSYSTEM HARDWARE

- Raspberry Pi 4 Model B
- Adafruit Accessories GPS Antenna 3-5V 28dB 5 meter SMA
- Adafruit Ultimate GPS HAT for Raspberry Pi A+/B+/Pi 2/3/4/Pi 5

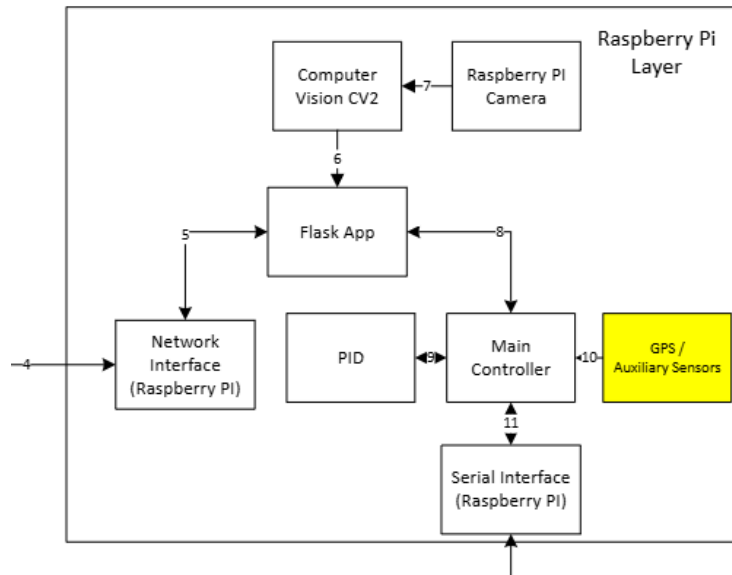


Figure 12: Example subsystem description diagram

4.10.2 SUBSYSTEM OPERATING SYSTEM

Raspberry Pi OS lite (64 - bit)

4.10.3 SUBSYSTEM SOFTWARE DEPENDENCIES

GPSD library [2]

4.10.4 SUBSYSTEM PROGRAMMING LANGUAGES

Python

4.10.5 SUBSYSTEM DATA STRUCTURES

Communication Protocol: Pi's UART at 9600 baud rate to send/receive data to and from the GPS module

4.10.6 SUBSYSTEM DATA PROCESSING

GPSD library functions [3]

4.11 RASPBERRY PI SUBSYSTEM 8 - SERIAL INTERFACE

The Serial Interface is the subsystem that allows the Raspberry Pi to communicate with the Arduino. This subsystem is responsible for allowing the Raspberry Pi to issue commands and receive sensor data to and from the Arduino.

4.11.1 SUBSYSTEM HARDWARE

Raspberry Pi 4B and Serial UART on the Raspberry Pi 4B

4.11.2 SUBSYSTEM OPERATING SYSTEM

Raspberry Pi OS

4.11.3 SUBSYSTEM SOFTWARE DEPENDENCIES

Raspberry Pi's USB Abstract Control Model Driver

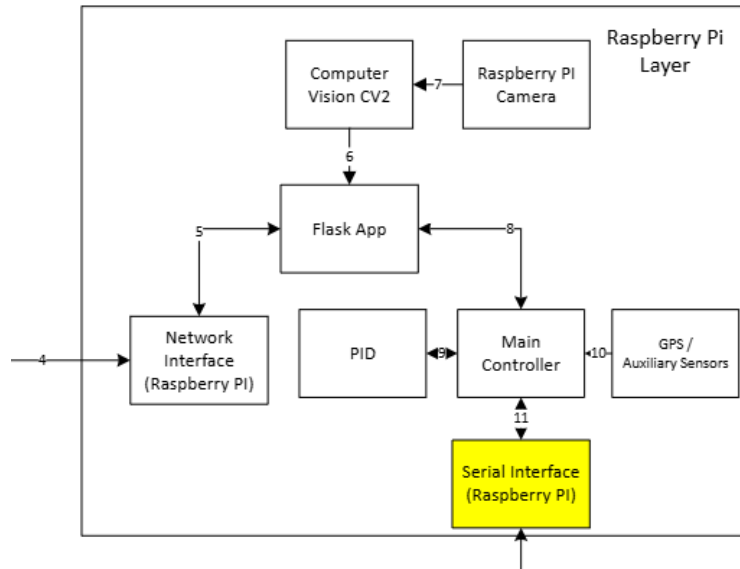


Figure 13: Serial Interface in the Raspberry Pi Layer

4.11.4 SUBSYSTEM PROGRAMMING LANGUAGES

Python Programming Language

4.11.5 SUBSYSTEM DATA STRUCTURES

The data structures used by this subsystem:

- Python Variables
- Python Dictionaries

4.11.6 SUBSYSTEM DATA PROCESSING

The data processing algorithms used are:

- ReadArduinoData
- WriteArduinoCommand
- UartReader
- ArduinoWriter
- handleData

The ReadArduinoData and WriteArduinoCommand are functions that interact with the Raspberry Pi's /dev/ttyACM0 to write data on the serial port at the baud rate of 115200. These are blocking functions that either read data or write data as a single string.

The UartReader and Arduino Writer are thread functions that are continuously running and allow the Raspberry Pi to continuously communicate with the Arduino.

The UartReader function continuously reads a single line by calling the ReadArduinoData function and

then passing the data to `handleData` function to parse and store.

The `arduinoWriter` function continuously writes values in the commands dictionary to the UART.

The `handleData` function parses the data for the format `key:value` and stores this in the `Blimp_data` dictionary.

5 ARDUINO LAYER SUBSYSTEMS

The Arduino layer communicates with the Raspberry Pi layer to send telemetry data and collect motor instruction data from the Pi. Additionally, this layer will also send the current battery percentage of the LiPo battery to the Pi. This section will dive deep into the specifics of how and what it uses to achieve the listed goals.

5.1 LAYER HARDWARE

ELEGOO MEGA R3 Board ATmega 2560

5.2 LAYER OPERATING SYSTEM

None: Uses Arduino IDE with a set of libraries.

5.3 LAYER SOFTWARE DEPENDENCIES

- Adafruit Sensors libraries [6] [1]
- SparkFun ICM 20948 library [5]
- Arduino Servo library
- Arduino I2C library

5.4 SUBSYSTEM 1 - SERIAL INTERFACE

The serial interface in the Arduino Layer is the subsystem that is responsible for communicating with the Raspberry Pi. This subsystem uses a UART on the Arduino's USB to communicate with the Pi.

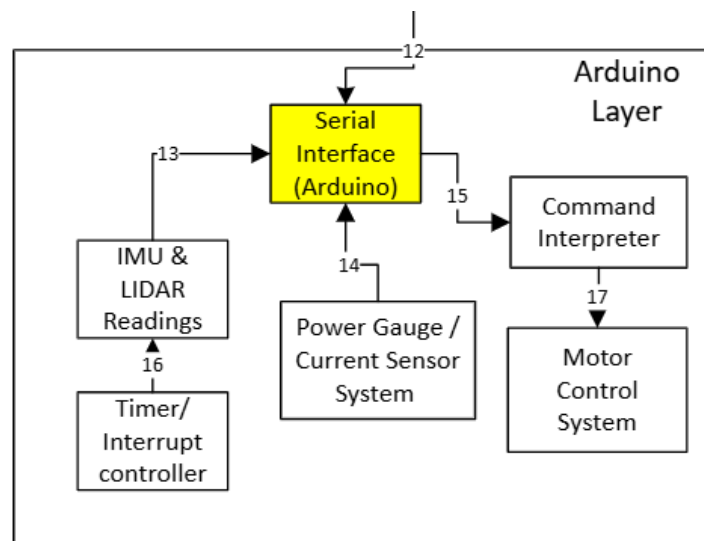


Figure 14: Serial Interface in teh Arduino Layer

5.4.1 SUBSYSTEM HARDWARE

UART on the Arduino Mega Board.

5.4.2 SUBSYSTEM OPERATING SYSTEM

NONE

5.4.3 SUBSYSTEM SOFTWARE DEPENDENCIES

Software Dependencies:

- Arduino Serial Library

5.4.4 SUBSYSTEM PROGRAMMING LANGUAGES

Arduino - C++/C

5.4.5 SUBSYSTEM DATA STRUCTURES

NONE

5.4.6 SUBSYSTEM DATA PROCESSING

This subsystem uses a simple polling algorithm to determine if data is available on the serial UART bus and then reads and passes that data to the command interpreter subsystem.

The data being sent to the Raspberry Pi is handled through an interrupt system that writes sensor data to the Pi.

5.5 SUBSYSTEM 2: IMU & LIDAR READINGS

The IMU and LIDAR Readings subsystem is a collection of external hardware sensors that retrieves raw data and formats the relevant data to provide orientation information and distance from an object for the Raspberry Pi. The data is periodically sent to the receiver at a fixed interval to improve overall system performance.

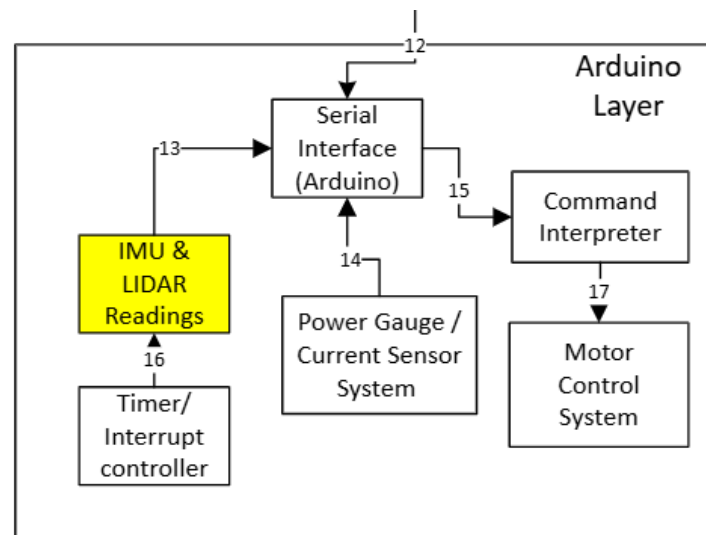


Figure 15: IMU & Lidar Readings subsystem diagram

5.5.1 SUBSYSTEM HARDWARE

- ELEGOO MEGA R3 Board ATmega 2560
- VL53L0X Lidar sensor
- ICM 20948 IMU sensor

5.5.2 SUBSYSTEM OPERATING SYSTEM

None: Uses the Arduino IDE with the sensor and communication libraries

5.5.3 SUBSYSTEM SOFTWARE DEPENDENCIES

- Adafruit Sensors libraries [6] [1]
- SparkFun ICM 20948 library [5]
- Arduino I2C library

5.5.4 SUBSYSTEM PROGRAMMING LANGUAGES

C/C++

5.5.5 SUBSYSTEM DATA STRUCTURES

Communication Protocol: I2C at 400 kHz with the microcontroller as the device master and the sensors as the device slaves

5.5.6 SUBSYSTEM DATA PROCESSING

Tilt-compensated Compass: Gyro offsets are found and applied to the gyroscope to calibrate it. Next, sufficient magnetometer and accelerometer data points in 3D are collected, and then calibrated using linear algebra techniques. The result will be matrices of both the magnetometer's and accelerometer's scale factors and offsets. Finally, the matrices are applied to real-time IMU readings to assist in calculating the heading, simulating a compass. [4]

5.6 SUBSYSTEM 3 - POWER GAUGE / CURRENT SENSOR

This subsystem uses a current sensor and a coulomb-counting algorithm to detect battery percentage. This system is responsible for letting the blimp know how much batter is remaining for the flight.

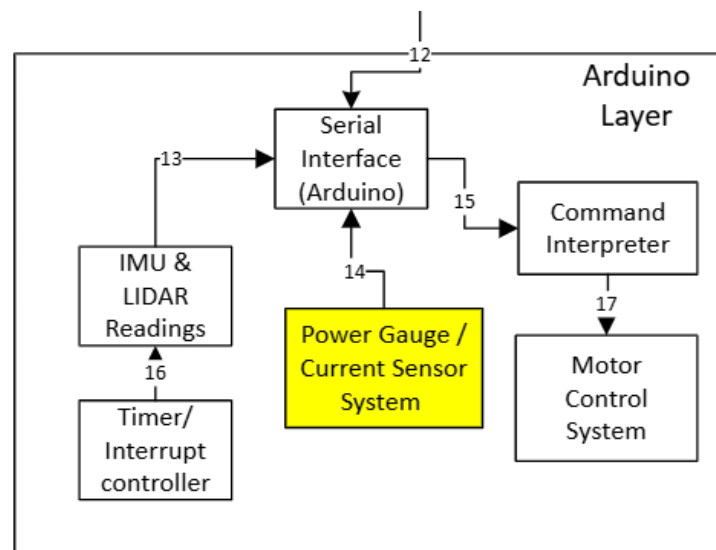


Figure 16: Example subsystem description diagram

5.6.1 SUBSYSTEM HARDWARE

ACS-758 HALL effect Current Sensor

5.6.2 SUBSYSTEM OPERATING SYSTEM

NONE

5.6.3 SUBSYSTEM SOFTWARE DEPENDENCIES

NONE

5.6.4 SUBSYSTEM PROGRAMMING LANGUAGES

Arduino Language - C/C++

5.6.5 SUBSYSTEM DATA STRUCTURES

C/C++ Variables

5.6.6 SUBSYSTEM DATA PROCESSING

This subsystem uses the coulomb-counting algorithm. This algorithm first detects the analog signal sent by the ACS-758 current sensor and then processes it by using known values such as the battery capacity, the resolution of the adc on the arduino, and the sensitivity of the current sensor.

First, the analog voltage is read then the raw voltage is converted to actual voltage by doing ADC calculations with the reference VCC and the adc resolution.

Then, the voltage is converted to amps using the sensitivity value of the current sensor. and then the current value is integrated using delta time to count the number of miliamp-hours used of the battery. The battery value out of hundred is derived as a fraction and then returned.

5.7 SUBSYSTEM 4 - COMMAND INTERPRETER

The command interpreter is the subsystem that is responsible for parsing commands being sent from the Pi to the Arduino. The command interpreter directs these commands the result of these commands to the motor control subsystem.

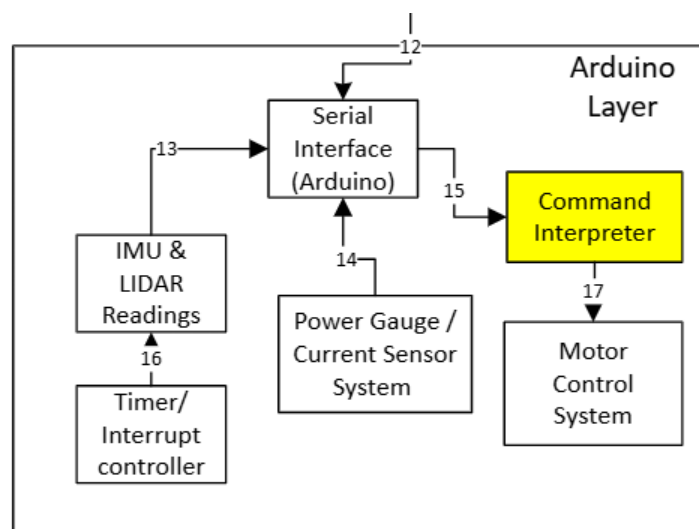


Figure 17: Command Interpreter subsystem in the Arduino Layer

5.7.1 SUBSYSTEM HARDWARE

NONE

5.7.2 SUBSYSTEM OPERATING SYSTEM

NONE

5.7.3 SUBSYSTEM SOFTWARE DEPENDENCIES

NONE

5.7.4 SUBSYSTEM PROGRAMMING LANGUAGES

Arduino Language - C++/C

5.7.5 SUBSYSTEM DATA STRUCTURES

C++/C variables C++/C string

5.7.6 SUBSYSTEM DATA PROCESSING

Raw string data from the Serial Interface is passed to the command interpreter. Which trims the message and then parses it for keywords that will allow the motor control system to control each individual motor speed.

5.8 SUBSYSTEM 5: TIMER/INTERRUPT CONTROLLER

The Timer/Interrupt Controller software subsystem is responsible for providing the IMU and LIDAR Readings subsystem with an interrupt signal that conveys when to send the sensor data to the Raspberry Pi.

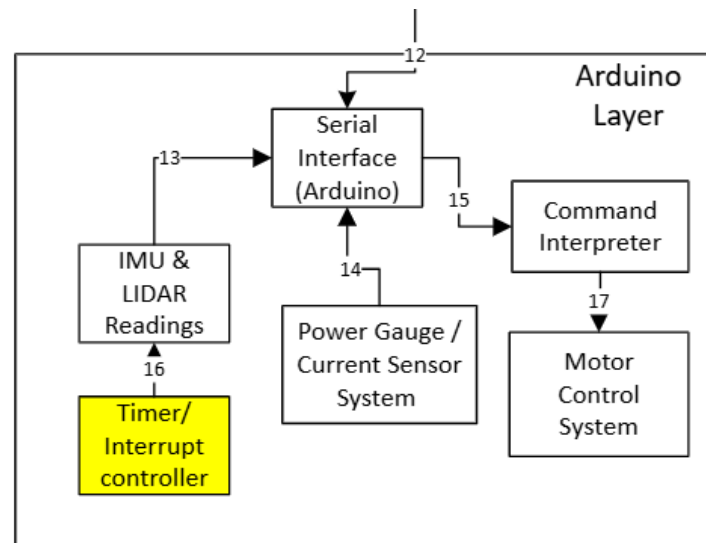


Figure 18: Timer/Interrupt subsystem diagram

5.8.1 SUBSYSTEM HARDWARE

ELEGOO MEGA R3 Board ATmega 2560: built-in timers

5.8.2 SUBSYSTEM OPERATING SYSTEM

None

5.8.3 SUBSYSTEM SOFTWARE DEPENDENCIES

Arduino standard core library

5.8.4 SUBSYSTEM PROGRAMMING LANGUAGES

C/C++

5.8.5 SUBSYSTEM DATA STRUCTURES

None

5.8.6 SUBSYSTEM DATA PROCESSING

None

5.9 SUBSYSTEM 6 - MOTOR CONTROL

The purpose of this subsystem is to control the speed of the motors on the airship. The motor speed will be controlled by using a PWM signal that is plugged in the ESCs which are plugged into the BLDC motors.

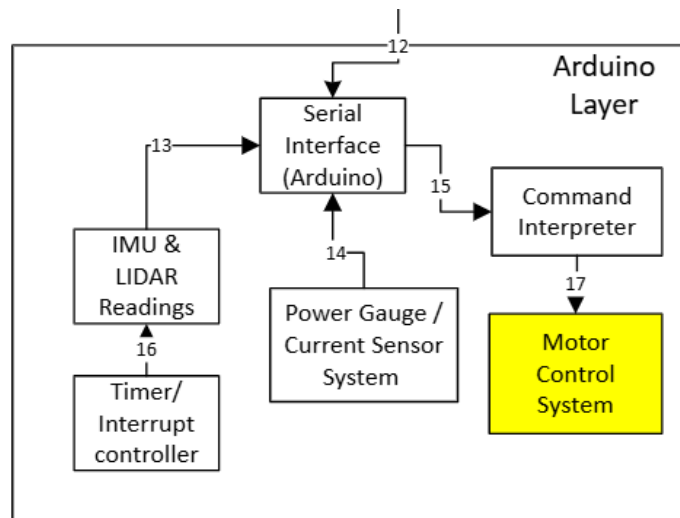


Figure 19: Motor Control subsystem in the Arduino Layer

5.9.1 SUBSYSTEM HARDWARE

- BLDC motors
- Electronic Speed Controllers

5.9.2 SUBSYSTEM OPERATING SYSTEM

NONE

5.9.3 SUBSYSTEM SOFTWARE DEPENDENCIES

Software Dependencies:

- Arduino - Servo Libraries

5.9.4 SUBSYSTEM PROGRAMMING LANGUAGES

Arduino Language - C/C++

5.9.5 SUBSYSTEM DATA STRUCTURES

Servo Library Objects

5.9.6 SUBSYSTEM DATA PROCESSING

The speed and motor values passed from the command interpreter are set to the servo objects corresponding to each motor.

6 APPENDIX A

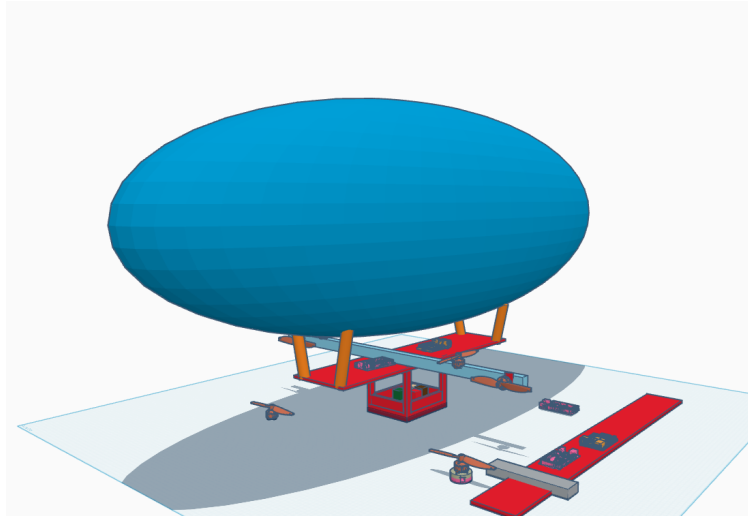


Figure 20: Blimp CAD Design

REFERENCES

- [1] Lady Ada. Adafruit VL53L0X Time of Flight MicroLIDAR Distance Sensor Breakout, 2024. Available at: <https://cdn-learn.adafruit.com/downloads/pdf/adafruit-vl53l0x-micro-lidar-distance-sensor-breakout.pdf> [Accessed: Sept. 2025].
- [2] Lady Ada. Adafruit Ultimate GPS HAT for Raspberry Pi, 2025. Available at: <https://cdn-learn.adafruit.com/downloads/pdf/adafruit-ultimate-gps-hat-for-raspberry-pi.pdf> [Accessed: Sept. 2025].
- [3] Martijn Braam. gpsd-py3, 2025. Available at: <https://github.com/MartijnBraam/gpsd-py3/blob/master/DOCS.md> [Accessed: Sept. 2025].
- [4] jremington. ICM 20948 AHRS. GitHub Repo, 2025. Available at: <https://github.com/jremington/ICM20948-AHRS> [Accessed : Sept.2025].
- [5] PaulZC. SparkFun ICM 20948 Arduino Library, 2025. Available at: https://github.com/sparkfun/SparkFun_ICM-20948_ArduinoLibrary [Accessed : Sept.2025].
- [6] Bryan Siepert. Adafruit TDK InvenSense ICM-20948 9-DoF IMU, 2024. Available at: <https://cdn-learn.adafruit.com/downloads/pdf/adafruit-tdk-invensense-icm-20948-9-dof-imu.pdf> [Accessed: Sept. 2025].